Stacks in Action — Expressions and Beyond

Lecture 07

Data Structures and Algorithms

Pranshu Mandal

Part 1: Expression Notation Systems

Why Do We Need Different Notations?

Humans write expressions in **Infix** notation: `A + B`

But what about `A + B * C`?

- Is it `(A + B) * C` or `A + (B * C)`?
- We need **operator precedence** rules and **parentheses** to disambiguate

The problem for computers:

- Parsing infix requires tracking precedence, associativity, and parentheses
- What if there was a notation with **no ambiguity at all**?

That's exactly what **Prefix** and **Postfix** notations provide!

Three Types of Notation

Notation	Format	Example (`A + B`)
Infix	Operator **between** operands	`A + B`
Prefix (Polish)	Operator **before** operands	`+ A B`
Postfix (Reverse Polish)	Operator **after** operands	`A B +`

Prefix — named after Polish logician Jan Łukasiewicz

Postfix — used in HP calculators, Java Virtual Machine, PostScript

Why Prefix/Postfix?

- **No parentheses needed** — precedence is inherent in the notation
- **No ambiguity** — can be evaluated in a single left-to-right scan
- **Easy for machines to evaluate** using a **Stack**

Operator Precedence and Associativity

Before converting expressions, we need these rules:

Operator	Precedence	Associativity
`^` (exponentiation)	Highest (3)	**Right-to-Left**
`*`, `/`	Medium (2)	Left-to-Right
`+`, `-`	Lowest (1)	Left-to-Right

Associativity matters for operators of equal precedence:

- `A - B - C` = `(A - B) - C` ← left-to-right
- `A ^ B ^ C` = `A ^ (B ^ C)` ← right-to-left

Conversion Examples

Infix	Prefix	Postfix
`A + B`	`+ A B`	`A B +`
`A + B * C`	`+ A * B C`	`A B C * +`
`(A + B) * C`	`* + A B C`	`A B + C *`
`A * B + C * D`	`+ * A B * C D`	`A B * C D * +`
`A ^ B ^ C`	`^ A ^ B C`	`A B C ^ ^`

Notice: In Postfix, the order of operands is preserved from Infix — only operators move!

Part 2: Infix → Postfix Conversion (Shunting-Yard Algorithm)

The Shunting-Yard Algorithm

Invented by **Edsger Dijkstra** — uses a **stack** for operators and outputs a postfix string.

Rules (scan left to right):

1. **Operand?** → Add directly to output
2. `(`? → Push onto stack
3. `)`? → Pop from stack to output until `(` is found; discard `(`
4. **Operator?** → Pop operators from stack to output while they have **higher or equal precedence** (for left-associative) or **strictly higher precedence** (for right-associative `^`), then push current operator
5. **End of expression?** → Pop all remaining operators to output

Traced Example 1: `A + B * C - D`

Step	Symbol	Stack	Output	Action
1	`A`		`A`	Operand → output
2	`+`	`+`	`A`	Push operator
3	`B`	`+`	`A B`	Operand → output
4	`*`	`+ *`	`A B`	`*` > `+`, push
5	`C`	`+ *`	`A B C`	Operand → output
6	`-`	`-`	`A B C * +`	Pop `*` (higher), pop `+` (equal), push `-`
7	`D`	`-`	`A B C * + D`	Operand → output
8	End		`A B C * + D -`	Pop all

Result: `A B C * + D -`

Traced Example 2: `(A + B) * (C - D)`

Step	Symbol	Stack	Output	Action
1	`(`	`(`		Push `(`
2	`A`	`(`	`A`	Operand → output
3	`+`	`( +`	`A`	Push (inside parentheses)
4	`B`	`( +`	`A B`	Operand → output
5	`)`		`A B +`	Pop until `(`; discard `(`
6	`*`	`*`	`A B +`	Push operator
7	`(`	`* (`	`A B +`	Push `(`
8	`C`	`* (`	`A B + C`	Operand → output
9	`-`	`* ( -`	`A B + C`	Push (inside parentheses)
10	`D`	`* ( -`	`A B + C D`	Operand → output

Traced Example 3: `A ^ B ^ C` (Right-Associative)

Step	Symbol	Stack	Output	Action
1	`A`		`A`	Operand → output
2	`^`	`^`	`A`	Push
3	`B`	`^`	`A B`	Operand → output
4	`^`	`^ ^`	`A B`	`^` is right-associative → do NOT pop equal precedence; push
5	`C`	`^ ^`	`A B C`	Operand → output
6	End		`A B C ^ ^`	Pop all

Result: `A B C ^ ^` which evaluates as `A ^ (B ^ C)` ✓

If `^` were left-associative, step 4 would pop the first `^`, giving `A B ^ C ^` = `(A ^ B) ^`

Brief Note: Infix → Prefix Conversion

The Trick: Convert to Prefix by reusing the Postfix algorithm!

1. **Reverse** the infix expression
2. **Swap** `(` and `)` in the reversed expression
3. Apply **Infix → Postfix** algorithm on the result
4. **Reverse** the output to get the Prefix expression

Quick Example: `(A + B) * C`

```
Step 1: Reverse          →  C * )B + A(
Step 2: Swap ( and )     →  C * (B + A)
Step 3: Convert to Postfix →  C B A + *
Step 4: Reverse          →  * + A B C
```

Prefix Result: `* + A B C` ✓

Part 3: Postfix Expression Evaluation

Postfix Evaluation Algorithm

Evaluating a Postfix expression is beautifully simple with a **stack**:

Rules (scan left to right):

1. **Operand?** → **Push** onto stack
2. **Operator?** → **Pop two** operands, apply operator, **push result** back
   - Second popped = left operand, First popped = right operand
   - `result = second_popped  OP  first_popped`
3. **End?** → Stack contains one element: **the final result**

Why does this work?

In Postfix, an operator always appears **immediately after** its operands — the stack naturally holds operands until their operator arrives.

Traced Example 1: `2 3 1 * + 9 -`

Step	Symbol	Stack	Action
1	`2`	`[2]`	Push operand
2	`3`	`[2, 3]`	Push operand
3	`1`	`[2, 3, 1]`	Push operand
4	`*`	`[2, 3]`	Pop 1 and 3 → `3 * 1 = 3` → Push 3
5	`+`	`[5]`	Pop 3 and 2 → `2 + 3 = 5` → Push 5
6	`9`	`[5, 9]`	Push operand
7	`-`	`[-4]`	Pop 9 and 5 → `5 - 9 = -4` → Push -4

Result: -4

Equivalent infix: `2 + (3 * 1) - 9 = -4` ✓

Traced Example 2: `5 1 2 + 4 * + 3 -`

Step	Symbol	Stack	Action
1	`5`	`[5]`	Push
2	`1`	`[5, 1]`	Push
3	`2`	`[5, 1, 2]`	Push
4	`+`	`[5, 3]`	Pop 2, 1 → `1+2=3` → Push
5	`4`	`[5, 3, 4]`	Push
6	`*`	`[5, 12]`	Pop 4, 3 → `3*4=12` → Push
7	`+`	`[17]`	Pop 12, 5 → `5+12=17` → Push
8	`3`	`[17, 3]`	Push
9	`-`	`[14]`	Pop 3, 17 → `17-3=14` → Push

Result: 14

Postfix Evaluation: C Code

```c
int evaluatePostfix(char* exp) {
    struct Stack s;
    s.top = -1;

    for (int i = 0; exp[i]; i++) {
        if (isdigit(exp[i])) {
            push(&s, exp[i] - '0');      // Convert char to int
        } else {
            int b = pop(&s);             // First popped = right operand
            int a = pop(&s);             // Second popped = left operand
            switch (exp[i]) {
                case '+': push(&s, a + b); break;
                case '-': push(&s, a - b); break;
                case '*': push(&s, a * b); break;
                case '/': push(&s, a / b); break;
            }
        }
    }
    return pop(&s);    // Final result
}
```

Time Complexity: O(n) — single pass through the expression.

Part 4: Depth-First Search (DFS) and Stacks

What is Depth-First Search?

A **graph traversal** algorithm that explores as far as possible along each branch before **backtracking**.

Analogy: Exploring a maze — you go deep down one path until you hit a dead end, then backtrack and try another path.

```
Consider this graph:

    A --- B --- E
    |     |
    C --- D --- F

DFS from A (one possible order): A → B → E → D → F → C
Goes DEEP first along A→B→E, backtracks, then goes A→B→D→F, etc.
```

DFS naturally uses a **Stack** — either explicitly, or via recursion's **call stack**.

DFS Algorithm Using an Explicit Stack

```
DFS(startNode):
    create empty Stack S
    push startNode onto S

    while S is not empty:
        node = pop(S)
        if node is not visited:
            mark node as visited
            process node
            push all UNVISITED neighbors of node onto S
```

Key insight: The stack ensures we always explore the **most recently discovered** node first — going deeper and deeper before backtracking.

DFS Trace Example

```
Graph:  A -- B -- E          DFS from A:
        |    |
        C -- D -- F

Step 1: Push A               Stack: [A]           Visited: {}
Step 2: Pop A, visit A       Stack: []            Visited: {A}
        Push neighbors B, C  Stack: [B, C]
Step 3: Pop C, visit C       Stack: [B]           Visited: {A, C}
        Push neighbor D      Stack: [B, D]
Step 4: Pop D, visit D       Stack: [B]           Visited: {A, C, D}
        Push neighbors B, F  Stack: [B, B, F]
Step 5: Pop F, visit F       Stack: [B, B]        Visited: {A, C, D, F}
Step 6: Pop B, visit B       Stack: [B]           Visited: {A, C, D, F, B}
        Push neighbor E      Stack: [B, E]
Step 7: Pop E, visit E       Stack: [B]           Visited: {A, C, D, F, B, E}
Step 8: Pop B, already visited → skip. Stack empty → DONE

DFS Order: A → C → D → F → B → E
```

DFS and Recursion: The Hidden Stack

The **recursive** version of DFS uses the **system call stack** as its stack:

```
void DFS_recursive(int node, int visited[], int adj[][MAX], int n) {
    visited[node] = 1;
    printf("%d ", node);

    for (int i = 0; i < n; i++) {
        if (adj[node][i] == 1 && !visited[i]) {
            DFS_recursive(i, visited, adj, n);   // recursive call = push
        }
    }
    // function returns = pop (backtrack)
}
```

Every recursive function call = push onto call stack

Every return from a function = pop from call stack

This is why Stacks and Recursion are fundamentally connected!

Key Takeaways

What We Learned Today

1. Expression Notation Systems

- Infix (human-friendly), Prefix (operator first), Postfix (operator last)
- Prefix and Postfix eliminate ambiguity — no parentheses needed

2. Infix → Postfix Conversion (Shunting-Yard Algorithm)

- Uses a stack for operators, outputs operands directly
- Handles precedence, associativity, and parentheses
- Infix → Prefix: reverse, swap brackets, convert to postfix, reverse again

3. Postfix Evaluation

- Push operands, pop two for each operator, push result — single pass O(n)

4. DFS and Stacks

- DFS goes deep before backtracking — naturally uses a stack
- Recursive DFS uses the call stack implicitly
- Stacks are the backbone of recursion, expression evaluation, and DFS

Review Questions

1. Convert the infix expression `(A + B) * C - D / E` to Postfix.
2. Convert `A * (B + C) / D` to Prefix using the reverse-and-convert trick.
3. Evaluate the postfix expression `6 2 3 + - 3 8 2 / + *` step by step.
4. In the Shunting-Yard Algorithm, what happens when a `)` is encountered?
5. Why does `^` (exponentiation) require special handling during Infix → Postfix conversion?
6. Trace DFS on the following graph starting from node 1:

```
1 -- 2 -- 4
|         |
3 ------- 5
```

7. Explain how every recursive function implicitly uses a stack.